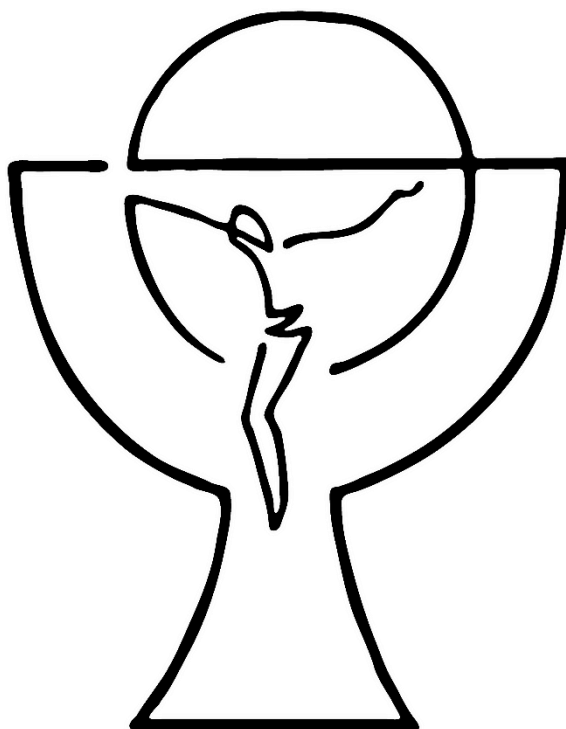


MCM – Modular Computing Market

Vizsgaremek



Készítette:

Bakk Hunor, Pásztor Simon, Halász Marcell

13.C

Szent István Katolikus Gimnázium és Technikum

2026

Tartalom

Bevezetés	5
A projekt célja	5
A rendszer felhasználói	6
A rendszer felépítése	6
MCM Frontend Dokumentáció	7
1. Bevezetés	7
2. Projekt Felépítése.....	8
3. Architektúra és Tervezési Elvek.....	9
4. Főbb komponensek és szolgáltatások.....	10
5. Navigáció és Útvonalak	12
6. UI Komponensek Viselkedése	13
7. Styling és téma	14
8. Fejlesztési és futtatási lépések.....	15
9. Funkciók működése.....	16
10. További fejlesztési tapasztalatok, technológiai alternatívák és jövőbeli lehetőségek	16
MCM Backend Dokumentáció.....	18
Bevezetés.....	18
Architektúra áttekintés	18
Mappastruktúra	19
Környezet és függőségek	20
index.js – a szerver indítása	20
Middleware-ek	21
Route-ok.....	22
auth.routes.js	22
protected.routes.js	23
Kontroller-ek (üzleti logika)	23
login.controller.js	23
register.controller.js.....	24
cart.controller.js.....	24
checkout.controller.js és order.controller.js	25
Middleware-ek	25
auth.middleware.js	25
optionalAuth.middleware.js	25
Modellek (Sequelize)	26

user.model.js	26
index.js (modellek inicializálása).....	26
Tipikus működési folyamat.....	26
Egyéb fontos részek	27
Tesztelés / futtatás.....	27
Jövőbeli bővítések.....	28
Példa kódrészletek a dokumentációhoz	28
Kosárba tétel meghívása	28
JWT ellenőrzése middleware-ben.....	29
Összefoglalás.....	29
MCM Adatbázis Dokumentáció	29
Bevezetés	29
Adatbázis Szerkezete - Áttekintés.....	30
Az egyes táblák részletes leírása.....	30
1. Users Tábla - Felhasználó Kezelés	30
2. Products Tábla - Termék Centrális Nyilvántartása	31
3. Users és Cart_Items Reláció - Vásárlási Kosár	33
4. Orders tábla - rendelések kezelése	34
5. OrderDetails tábla - rendelés sorok	36
6. GameKeys Tábla - videójáték kulcsok nyilvántartása	38
7. GiftCards tábla - ajándékkártyák kezelése	39
Adatbázis Relációk Összefoglalása.....	40
Teljes Rendelés (Order + Details + Product Info)	41
Felhasználó vásárlási története.....	42
Készlet nyilvántartás (hardware + kulcsok + kártyák).....	42
Teljesítmény optimalizálás és indexek.....	43
Biztonsági megjegyzések	43
Jelenlegi adatok összefoglalása.....	43
Források.....	44
Technológiák és Keretrendszer	44
Backend:.....	44
Frontend:.....	44
Adatbázis:	44
Programozási Nyelvek	44
Fejlesztési Eszközök és Könyvtárak	44
Backend Dependencies:	44

Frontend Dependencies:.....	45
Fejlesztési Eszközök:.....	45
Projekt Struktúra és Konfiguráció	45
Backend Struktúra:.....	45
Frontend Struktúra:.....	45
Környezeti Konfiguráció	45
Verziókezelés és Deployment	46

Bevezetés



A projekt célja

A projekt célja egy modern, digitális termékeket értékesítő webáruház létrehozása, amely videojátékokat, digitális kulcsokat és különböző ajándékkártyákat kínál a felhasználók számára. A rendszer célja, hogy gyors, biztonságos és felhasználóbarát vásárlási élményt biztosítson, amely a termékek böngészésétől a kosár kezelésén át egészen a rendelés leadásáig minden lépést lefed.

A fejlesztés során fontos szempont volt a valós webáruházak működésének modellezése, a stabil adatbázis-struktúra kialakítása, valamint a modern webes technológiák alkalmazása. A rendszer három fő komponensből épül fel: Angular alapú frontend, Node.js + Express alapú backend, valamint MySQL adatbázis, amely XAMPP környezetben fut.

A projekt célja továbbá, hogy bemutassa a frontend, backend és adatbázis rétegek együttműködését, és egy olyan működőképes rendszert hozzon létre, amely megfelel a vizsgakövetelményeknek.

A webáruház fő céljai a következők:

- digitális termékek (játékok, kulcsok, giftcardok) értékesítése
- egyszerű és gyors vásárlási folyamat biztosítása
- kosár funkció és rendeléskezelés megvalósítása
- szállítási adatok rögzítése és tárolása
- felhasználói regisztráció és bejelentkezés kezelése
- digitális kulcsok automatikus kiosztása a rendelés után
- a backend és frontend közötti kommunikáció REST API-n keresztül

A rendszer célja egy olyan webáruház létrehozása, amely valós működést modellez, és technikailag is megfelelő alapot biztosít a további fejlesztésekhez.

A rendszer felhasználói

A webáruházat többféle felhasználó veszi igénybe, és a rendszer különböző jogosultsági szinteket biztosít a vásárlási folyamat során. A vásárlók böngészhetik a termékeket, kosárba helyezhetik őket, majd leadhatják a rendelést. A rendszer kétféle vásárlási módot különböztet meg: a hardver termékek megvásárlásához kötelező a bejelentkezés, mivel ezek fizikai termékek, amelyekhez személyes és szállítási adatok szükségesek, és a rendelést a felhasználói fiókhoz kell kötni. A digitális termékek, például a játék kulcsok és ajándékkártyák megvásárlásához viszont nem szükséges bejelentkezni, mivel ezek elektronikusan kerülnek átadásra, és a vásárlás vendégként is elvégezhető.

A regisztrált felhasználók be tudnak jelentkezni, és a rendszer a fiókjukhoz köti a rendeléseiket. Számukra gyorsabb vásárlási folyamat érhető el, valamint lehetőségük van a korábbi rendelések megtekintésére és a hardver termékek megvásárlására. A bejelentkezési funkció a backendben már tesztelve működik.

A rendszer opcionálisan adminisztrátorokat is támogathat, akik kezelhetik a termékeket, készleteket, digitális kulcsokat és a rendeléseket. Bár jelenleg nincs külön admin felület, a rendszer felépítése lehetővé teszi a későbbi bővítést.

A webáruház elsődleges célcsoportját a digitális játékokat és ajándékkártyákat vásárló felhasználók, valamint azok alkotják, akik hardver termékeket szeretnének rendelni biztonságos, regisztrált fiókkal.

A rendszer felépítése

A webáruház három fő technológiai rétegből épül fel: a frontendből, a backendből és az adatbázisból. A frontend Angular keretrendszerre épül, amely biztosítja a felhasználói felület megjelenítését, a termékek böngészését, a kosár kezelését és a rendelési folyamat lépéseit. A felhasználó minden művelete ezen a felületen keresztül történik, amely modern, reszponzív és könnyen kezelhető kialakítást kapott.

A backend Node.js és Express alapokra épül, és a rendszer logikai működéséért felel. Ez a réteg kezeli a bejelentkezést, a termékek lekérdezését, a kosár műveleteit, a rendelés feldolgozását, valamint a digitális kulcsok kiosztását. A frontend és a backend között REST API biztosítja a kommunikációt, amely JSON formátumban továbbítja az adatokat.

A rendszer harmadik eleme a MySQL adatbázis, amely a termékeket, felhasználókat, rendeléseket, digitális kulcsokat és minden egyéb szükséges információt tárol. Az adatbázis XAMPP környezetben fut, és stabil alapot biztosít a webáruház működéséhez.

A backend közvetlenül kapcsolódik az adatbázishoz, így minden adatmozgás biztonságosan és megbízhatóan történik.

Ez a három réteg együttműködve alkotja a teljes rendszert, amely jól strukturált, könnyen bővíthető és a modern webes követelményeknek megfelelő megoldást nyújt.

MCM Frontend Dokumentáció

1. Bevezetés

A SMH-Projekt frontend egy Angular alapú webalkalmazás, amely egy modern webáruház funkcióit valósítja meg. A projekt célja, hogy a felhasználók számára gyors, biztonságos és átlátható vásárlási élményt nyújtson, miközben a fejlesztők számára is jól strukturált, skálázható és karbantartható kódot biztosít.

Motiváció és célcsoport:

A webáruház célcsoportja a digitális termékek és hardverek iránt érdeklődő felhasználók, akik számára fontos a gyors böngészés, egyszerű kosárkezelés, biztonságos fizetés és átlátható rendelési folyamat. Az üzleti igények között szerepel a responszív design, a valós idejű termékfrissítés, a felhasználói hitelesítés, valamint a jövőbeli bővítési lehetőségek (pl. ajánlórendszer, admin felület, új fizetési módok).

Technológiai döntések:

Az Angular választása mellett szólt a moduláris felépítés, a fejlett CLI eszközök, a TypeScript alapú fejlesztés, valamint a nagyvállalati szintű támogatás és dokumentáció. Alternatívák lehettek volna React vagy Vue, de az Angular előnye a beépített routing, formkezelés, dependency injection és a komplexebb projektekhez ideális architektúra. A projektben az Angular CLI-t használtam a gyors komponens-, szolgáltatás- és modulgeneráláshoz.

Fejlesztési workflow:

A fejlesztés során a csapatmunka támogatására git verziókezelést, feature branch workflow-t, pull requesteket és CI/CD pipeline-t alkalmaztam. A tesztelés egységtesztekkel és integrációs tesztekkel történik, az automatizált build és deploy folyamatok biztosítják a gyors iterációt és a hibák korai kiszűrését. A fejlesztési környezetben proxy-t használok, így a frontend és backend közötti kommunikáció egyszerűen tesztelhető.

Modern frontend trendek:

A projektben kiemelt szerepet kapott a reszponzív design, a felhasználói élmény (UX), az accessibility (akadálymentesség), valamint a performance optimalizálás. A UI komponensek kapszulázottak, újrahasznosíthatók és tesztelhetők. A styling SCSS-ben történik, a custom-theme.scss-ben definiált színpaletta és változók lehetővé teszik a gyors arculatváltást.

Mappastruktúra és CLI példák:

Az Angular CLI segítségével a projekt mappastruktúrája logikusan elkülöníti az alkalmazás fő részeit. Például:

```
```shell
ng new frontend --routing --style=scss
ng generate module features/product --route product --module app
ng generate component shared/header
ng generate service services/cart
```
```

A `src/app` mappa tartalmazza a fő alkalmazás logikát, komponenseket és szolgáltatásokat. Az `src/assets` mappában találhatók a képek és logók, amelyek a vizuális megjelenést támogatják. A globális stílusokat a `src/styles.css` fájlban definiáltam, míg az egyedi témát a `src/custom-theme.scss`-ben. A proxy beállításokat a `proxy.conf.json`-ban kezeltem, hogy a fejlesztés során a frontend és backend közötti kommunikáció zökkenőmentes legyen.

A kulcsfájlok között szerepel az útvonalakat kezelő `app.routes.ts`, a konfigurációkat tartalmazó `app.config.ts`, a HTTP kérések biztonságát növelő `core/auth.interceptor.ts`, valamint a szolgáltatásokat, újrahasznosítható komponenseket és funkcionális modulokat tartalmazó mappák. Ezek a struktúrák segítik a projekt átláthatóságát és bővíthetőségét.

2. Projekt Felépítése

A projekt szerkezete támogatja a moduláris fejlesztést és a könnyű bővíthetőséget. A komponensek, szolgáltatások és modulok logikusan elkülönülnek egymástól, így a fejlesztés,

karbantartás és tesztelés egyszerűbb. A kulcsfájlok, mint az útvonalakat kezelő ``app.routes.ts``, a konfigurációkat tartalmazó ``app.config.ts``, valamint a biztonságot növelő ``core/auth.interceptor.ts`` jól elkülönülnek, így a projekt átlátható és skálázható. A globális és egyedi stílusok, valamint a proxy beállítások külön mappákban és fájlokban találhatóak, ezzel támogatva a fejlesztési workflow-t és a rezponzív, egységes megjelenést.

3. Architektúra és Tervezési Elvek

Az alkalmazás architektúrája moduláris, több fő Angular modulból épül fel: core, shared, features és app modul. A core modul tartalmazza az alapvető szolgáltatásokat (auth, interceptor, config), a shared modulban helyezkednek el az újrahasznosítható komponensek (header, cart-drawer, toast), míg a features modulok a funkcionális egységeket (pl. termékek, rendelés, profil) fogják össze. Az app modul a fő bootstrap és routing logikát tartalmazza.

A komponens hierarchia logikusan épül fel: a fő layout komponens tartalmazza a fejléct, oldalsó drawer-t, és a router outlet-et. A state management szolgáltatásokban történik, főként BehaviorSubject és Observable mintákkal, így a komponensek automatikusan frissülnek az adatok változásakor. A dependency injection (DI) lehetővé teszi, hogy a szolgáltatások singletonként működjenek, így minden komponens ugyanazt az állapotot látja.

Az architektúra támogatja a lazy loading-ot, így a nagyobb feature modulok csak akkor töltődnek be, amikor szükséges. Ez javítja a teljesítményt és csökkenti a bundle méretet. Az API integráció HttpClient-tel történik, az AuthInterceptor automatikusan hozzáfűzi a JWT tokent minden kéréshez. Az architektúra diagram (leírás):

- AppModule
 - CoreModule (auth, config, interceptor)
 - SharedModule (header, cart-drawer, toast)
 - FeaturesModule (product, cart, order, profile)

A komponensek, szolgáltatások és modulok szigorúan elkülönülnek, így a projekt könnyen bővíthető és karbantartható.

Az alkalmazás architektúrája a modern Angular best practice-eket követi. A modulok (Core, Shared, Features, App) egymástól függetlenül fejleszthetők, így a csapatmunka hatékonyabb. A CoreModule singleton szolgáltatásokat tartalmaz, például AuthService, ConfigService, ErrorHandlerService, amelyek globálisan elérhetők. A SharedModule újrahasznosítható UI elemeket, pipe-okat, direktívákat, utility függvényeket tartalmaz. A FeaturesModule minden üzleti funkciót különálló modulba szervez, például ProductModule, CartModule, OrderModule, ProfileModule.

A komponens hierarchia a következő:

- AppComponent (root)
- HeaderComponent
- CartDrawerComponent
- RouterOutlet
- ToastComponent

A state management a szolgáltatásokban történik, de nagyobb projektekben bevezethető NgRx vagy Akita. A dependency injection lehetővé teszi a mock szolgáltatások használatát teszteléskor. Az architektúra támogatja a lazy loading-ot, route guard-okat, resolver-eket, interceptorokat, így a biztonság, teljesítmény és UX optimalizált.

4. Főbb komponensek és szolgáltatások

A termékek listázásához ProductService-t használunk, amely Observable formában adja vissza a termékeket. A termékek megjelenítéséhez ProductListComponent és ProductCardComponent felel, a részletező ProductDetailComponent dinamikusan tölti be az adatokat route paraméter alapján. A CartService a kosár állapotát kezeli, minden kosárművelet (hozzáadás, törlés, módosítás) automatikusan frissíti az állapotot.

A komponensek interface-eket használnak, pl. Product, CartItem, Order, így a típusbiztonság garantált. A szolgáltatások tesztelhetők Angular TestBed-del, mock API-val, így a hibák gyorsan kiszűrhetők. Példa interface:

```
``typescript
```

```
export interface Product {
  id: string;
  name: string;
  price: number;
  image: string;
  description: string;
}
'''
```

A toast értesítések ToastService segítségével jelennek meg, rövid visszajelzést adva a felhasználónak. Az AuthService kezeli a bejelentkezést, regisztrációt, token tárolást, és a felhasználói állapotot. A szolgáltatások singletonként működnek, így minden komponens ugyanazt az állapotot látja.

A komponensek fejlesztése során figyeltem a tesztelhetőségre, kapszulázásra, újrahasznosíthatóságra és a rezponzív designra. Angular Material vagy custom UI elemek használhatók, a design rendszer könnyen bővíthető.

A ProductService felel a termékek lekérdezéséért, szűréséért, rendezéséért. A CartService a kosár állapotát kezeli, minden művelet után event emitterrel értesíti a UI-t. Az OrderService a rendelési folyamatot, fizetési integrációt, rendelési előzményeket kezeli. Az AuthService a bejelentkezést, regisztrációt, token kezelést, jogosultságokat, profil frissítést valósítja meg.

A komponensek fejlesztése során figyeltem a kapszulázásra, input/output property binding-re, event emitterre, observer pattern-re. Példa ProductListComponent:

```
```typescript
@Component({ selector: 'app-product-list', templateUrl: './product-list.component.html' })
export class ProductListComponent implements OnInit {
 products$: Observable<Product[]>;
 constructor(private productService: ProductService) {}
 ngOnInit() {
 this.products$ = this.productService.getProducts();
 }
}
```

```
}
...

```

A `CartDrawerComponent` animációval jelenik meg, a kosár tartalma automatikusan frissül. A `ToastComponent` rövid visszajelzéseket jelenít meg, a hibák, sikeres műveletek vagy információk gyors kommunikációjához.

## 5. Navigáció és Útvonalak

Az Angular Router-rel valósítottam meg az útvonalak kezelését. Az `app.routes.ts`-ben definiáltam a fő útvonalakat, a feature modulokhoz lazy loading-ot alkalmaztam. Route guard-ok (`AuthGuard`) biztosítják, hogy csak bejelentkezett felhasználók érhetik el a védett oldalakat. Route resolver-ekkel előtöltöm a szükséges adatokat, így a komponensek csak akkor jelennek meg, ha minden adat rendelkezésre áll.

Az útvonalak paraméterezhetők, pl. termék részletező oldal: ``/products/:id``. A navigáció animációkkal, loading indikátorokkal, breadcrumb-mal és progress bar-ral bővíthető. Példa route resolver:

```
``typescript
@Injectable({ providedIn: 'root' })
export class ProductResolver implements Resolve<Product> {
 constructor(private productService: ProductService) {}
 resolve(route: ActivatedRouteSnapshot): Observable<Product> {
 return this.productService.getProductById(route.paramMap.get('id'));
 }
}
...

```

A navigáció best practice: minden oldal elérhető legyen URL-ből, a visszalépés, előrelépés, deep linking támogatott. A routing tesztelhető Angular Router Testing Module-lal.

Az Angular Router lehetővé teszi a modulok lazy loading-ját, route guard-okat, resolver-eket, paraméterezett útvonalakat. Példa routing konfiguráció:

```

```typescript
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'products', loadChildren: () => import('./features/product/product.module').then(m =>
m.ProductModule) },
  { path: 'cart', component: CartDrawerComponent, canActivate: [AuthGuard] },
  { path: 'profile', component: ProfileComponent, canActivate: [AuthGuard] },
  { path: '**', component: NotFoundComponent }
];
```

```

A navigáció UX szempontból animációkkal, progress bar-ral, breadcrumb-mal, loading indikátorokkal bővíthető. A deep linking, visszalépés, előrelépés, history támogatott. Route resolver-ekkel előtöltöm a szükséges adatokat, így a felhasználó nem találkozik üres vagy hibás oldallal.

## 6. UI Komponensek Viselkedése

A UI komponensek fejlesztése során kiemelt figyelmet fordítottam a felhasználói élményre, accessibility-re és interaktivitásra. A fejléc (HeaderComponent) tartalmazza a navigációt, keresőt, kosár ikont, profil linket. A kosár drawer (CartDrawerComponent) animáltan jelenik meg, a tartalom automatikusan frissül az állapotváltozáskor.

A termék kártyák (ProductCardComponent) vizuálisan kiemelik a termékeket, a kosárba helyezés gomb intuitív és gyors. A termékek listázása lapozással, szűréssel, rendezéssel történik. A UI komponensek kapszulázottak, újrahasznosíthatók, tesztelhetők. Angular Material vagy custom UI elemek használhatók.

Accessibility szempontok: minden gomb, input, lista elérhető billentyűzettel és képernyőolvasóval. ARIA attribútumok, alt szövegek, kontraszt optimalizálás, tab indexek biztosítják az akadálymentességet. Animációk (Angular Animations, CSS transitions) javítják a felhasználói élményt, pl. drawer nyitás/zárás, toast megjelenés/eltűnés.

A UI komponensek tesztelése egységtesztekkel, snapshot tesztekkel, felhasználói tesztekkel történik. Design pattern-ek: kapszulázás, observer pattern, event emitter, input/output property binding. A UI komponensek könnyen bővíthetők tooltip-ekkel, modális ablakkal, progress bar-ral, skeleton loading-gal.

A UI komponensek fejlesztése során figyeltem az accessibility-re: minden gomb, input, lista elérhető billentyűzettel és képernyőolvasóval. ARIA attribútumok, alt szövegek, kontraszt optimalizálás, tab indexek biztosítják az akadálymentességet. Animációk (Angular Animations, CSS transitions) javítják a felhasználói élményt, pl. drawer nyitás/zárás, toast megjelenés/eltűnés.

A termék kártyák (ProductCardComponent) vizuálisan kiemelik a termékeket, a kosárba helyezés gomb intuitív és gyors. A termékek listázása lapozással, szűréssel, rendezéssel történik. A UI komponensek kapszulázottak, újrahasznosíthatók, tesztelhetők. Angular Material vagy custom UI elemek használhatók.

A UI komponensek tesztelése egységtesztekkel, snapshot tesztekkel, felhasználói tesztekkel történik. Design pattern-ek: kapszulázás, observer pattern, event emitter, input/output property binding. A UI komponensek könnyen bővíthetők tooltip-ekkel, modális ablakkal, progress bar-ral, skeleton loading-gal.

## 7. Styling és téma

A custom-theme.scss-ben egyedi színpalettát, változókat és mixineket definiáltam, amelyek lehetővé teszik a gyors stílusváltást és a design rendszer bővítését. A styles.css-ben globális stílusokat, tipográfiát és reszponzív beállításokat helyeztem el. A reszponzív design biztosítja, hogy az alkalmazás mobilon, tableten és asztali gépen is jól használható legyen. A stílusok kapszulázása segíti a karbantartást és a bővítést.

### Példa SCSS:

```
````scss
```

```
$primary: #1976d2;
```

```
$accent: #ff4081;
$warn: #f44336;
body {
  font-family: 'Roboto', sans-serif;
  background: #f5f5f5;
}
````
```

A styling fejlesztése során figyeltem a modern design trendekre, például Material Design vagy Flat Design elemekre. A színpaletta könnyen módosítható, így a projekt arculata gyorsan frissíthető. A stílusok kapszulázása segíti a komponensek újrahasznosítását és a karbantartást. A rezponzív design mellett dark mode és accessibility funkciók is bevezethetők, így minden felhasználó számára optimális élményt nyújt az alkalmazás.

## 8. Fejlesztési és futtatási lépések

A fejlesztés első lépése a projekt alapjainak lefektetése volt. Az Angular CLI segítségével hoztam létre az alkalmazást, majd a mappastruktúrát úgy alakítottam ki, hogy a komponensek, szolgáltatások és modulok jól elkülönüljenek egymástól. Ez a felosztás segítette a későbbi bővítést és karbantartást. A backend API integrációja során először a proxy.conf.json-t konfiguráltam, hogy a fejlesztés alatt a frontend és backend közötti kommunikáció egyszerűen működjön. Ez lehetővé tette, hogy az Angular alkalmazásból közvetlenül hívjam a backend végpontokat, anélkül, hogy CORS problémákba ütköznék.

### Fejlesztési lépések:

- `git clone <repo>`
- `cd frontend && npm install`
- `npm start` (proxy.conf.json beállítva)

A CI/CD pipeline automatizálja a build, teszt és deploy folyamatokat, így a fejlesztési ciklus gyorsabb és megbízhatóbb. A fejlesztési környezetben proxy-t használok, így a frontend és backend közötti kommunikáció egyszerűen tesztelhető.

## 9. Funkciók működése

A termék-szolgáltatás azért készült, hogy a termékek adatai mindig naprakészek legyenek, és a felhasználó valós időben láthassa a változásokat. Az Observable alapú adatkezelés lehetővé teszi, hogy a komponensek automatikusan frissüljenek, ha új termék kerül a rendszerbe vagy módosul egy meglévő.

A kosár állapotkezelő szolgáltatás célja, hogy a kosár tartalma minden komponens számára elérhető legyen, és a változások automatikusan frissüljenek. Ez a megközelítés biztosítja, hogy a felhasználó mindig a legfrissebb kosáradatokat látja, függetlenül attól, hogy melyik oldalon tartózkodik.

Az autentikációs szolgáltatás a felhasználói hitelesítés és jogosultságkezelés központi eleme. A JWT tokenek használata biztonságos és skálázható megoldást nyújt, mivel a tokenek automatikusan hozzáadódnak az API kérésekhez, így a backend felismeri a felhasználót és engedélyezi a védett végpontokat.

A toast értesítések célja, hogy a felhasználó gyors visszajelzést kapjon a műveletekről. A rövid, automatikusan eltűnő üzenetek nem zavarják a felhasználót, de mégis biztosítják a visszajelzést.

A styling és téma kialakítása során a custom-theme.scss-ben definiált színpaletta és változók lehetővé teszik a gyors stílusváltást és a design rendszer bővítését. A globális stílusok biztosítják az egységes megjelenést, a reszponzív design pedig garantálja, hogy az alkalmazás minden eszközön jól használható legyen.

## 10. További fejlesztési tapasztalatok, technológiai alternatívák és jövőbeli lehetőségek

A projekt jövőbeli fejlesztési lehetőségeit röviden összefoglalva: a felhasználói profil bővítése, rendelési előzmények megjelenítése, új fizetési módok integrálása, admin felület létrehozása, mesterséges intelligencia alapú ajánlórendszer, valós idejű chat funkció, big data elemzés, felhő



alapú infrastruktúra, marketing és SEO optimalizálás. A technológiai alapok lehetővé teszik, hogy az alkalmazás könnyen bővíthető és karbantartható legyen.

#### Technológiai alternatívák:

Az Angular mellett alternatívák lehetnek React, Vue, Svelte, Next.js vagy Nuxt. A React előnye a flexibilis komponensmodell és hatalmas ökoszisztéma, míg a Vue egyszerűbb tanulási görbével és gyors prototípus-készítéssel bír. Svelte modern, compiler-alapú megközelítést alkalmaz, így kisebb bundle méretet és gyorsabb renderelést biztosít. Next.js és Nuxt SSR-t és statikus generálást támogatnak, ami SEO és performance szempontból előnyös lehet. Az Angular választása a nagyvállalati támogatás, beépített DI, routing és formkezelés miatt volt indokolt.

#### Fejlesztési tapasztalatok:

A projekt során kiemelt figyelmet fordítottam a hibakezelésre, state managementre (BehaviorSubject, Observable), API integrációra (HttpClient, interceptor), performance tuningra (lazy loading, optimalizált asset kezelés), accessibility-re (ARIA, keyboard navigation), és UX-re (animációk, loading indikátorok, toast értesítések). A tesztelés egységes tesztekkel, integrációs tesztekkel és e2e tesztekkel történik, a CI/CD pipeline automatikusan építi és deployolja az alkalmazást.

#### Jövőbeli lehetőségek:

- Microfrontend architektúra: több csapat párhuzamos fejlesztése, független deploy.
- PWA (Progressive Web App): offline működés, push értesítések, installálhatóság.
- Real-time funkciók: WebSocket, SignalR, valós idejű chat, értesítések.
- Cloud deployment: Docker, Kubernetes, felhőszolgáltatók (AWS, Azure, GCP).
- Automatizált tesztelés: Jest, Cypress, Playwright, SonarQube.
- Monitoring és logging: Sentry, LogRocket, Google Analytics, Prometheus.
- Rollback és verziókezelés: feature flag-ek, blue-green deployment.
- Marketing és SEO: meta tag-ek, sitemap, structured data, analitika.
- Felhasználói visszajelzés: feedback modul, A/B tesztelés, heatmap.

#### Biztonsági és skálázhatósági szempontok:

A JWT tokenek, route guard-ok, input validáció, XSS/CSRF elleni védelem, HTTPS használata, rate limiting, audit logok mind hozzájárulnak a biztonságos működéshez. A

skálázhatóságot a moduláris felépítés, cache-elés, CDN, optimalizált asset kezelés és cloud infrastruktúra támogatja.

#### DevOps és deployment:

A CI/CD pipeline automatikusan építi, teszteli és deployolja az alkalmazást. A Docker konténerizáció, Kubernetes orchestration, monitoring és logging eszközök (pl. Sentry, Prometheus) segítik a stabil működést. Rollback lehetőségek, feature flag-ek és blue-green deployment biztosítják a gyors hibajavítást és a folyamatos fejlesztést. Összefoglalva, a SMH-Projekt frontend technológiai alapjai, fejlesztési workflow-ja és architektúrája lehetővé teszik a jövőbeli bővítést, skálázást és biztonságos működést. A modern frontend trendek, best practice-ek és DevOps eszközök alkalmazása biztosítja, hogy az alkalmazás hosszú távon is versenyképes és karbantartható maradjon. A részletes dokumentáció, tesztelési stratégia, fejlesztési workflow, architektúra, UI/UX, accessibility, performance, DevOps, deployment, monitoring, rollback, SEO, analitika, marketing, felhasználói visszajelzés, jövőbeli lehetőségek, technológiai alternatívák, best practice-ek, skálázhatóság, biztonság mind hozzájárulnak a sikeres, modern, professzionális webalkalmazás fejlesztéséhez.

## **MCM Backend Dokumentáció**

### **Bevezetés**

Az alábbi dokumentáció a Modular Computing Market Projekt-remek backendjét ismerteti részletesen. A leírás átfogó: bemutatja az alkalmazás felépítését, a fontosabb modulokat, a mappastruktúrát és a működést. A dokumentáció nem tér ki a tényleges adatbázis-sémára vagy a mcm.sql file-ra – csak a szerver oldali logikára, útvonalakra, middleware-ekre és a kapcsolódó kódra.

### **Architektúra áttekintés**

A backend Node.js/Express alapon működik, Sequelize ORM-mel kapcsolódva egy SQL adatbázishoz. Az alkalmazás logikailag rétegezett:

- index.js – belépési pont, middleware/tűrések beállítása
- routes/ – REST-es végpontok gyűjteménye
- controllers/ – üzleti logika a route-ok számára
- models/ – Sequelize modellek (entitások)
- middleware/ – hitelesítés, opcionális auth, egyéb feldolgozás
- uploads/ – statikus fájlok kiszolgálása

## Mappastruktúra

backend/

- ├─ index.js
- ├─ src/
  - │ └─ config/
    - │ │ └─ config.json
  - │ └─ controllers/
    - │ │ └─ auth.controller.js
    - │ │ └─ cart.controller.js
    - │ │ └─ checkout.controller.js
    - │ │ └─ login.controller.js
    - │ │ └─ order.controller.js
    - │ │ └─ register.controller.js
  - │ └─ middleware/
    - │ │ └─ auth.middleware.js
    - │ │ └─ optionalAuth.middleware.js
  - │ └─ models/
    - │ │ └─ index.js
    - │ │ └─ cartitem.model.js
    - │ │ └─ gamekey.model.js
    - │ │ └─ giftcard.model.js
    - │ │ └─ order.model.js
    - │ │ └─ orderdetail.model.js
    - │ │ └─ product.model.js
    - │ │ └─ user.model.js
  - │ └─ routes/
    - │ │ └─ auth.routes.js
    - │ │ └─ cart.routes.js
    - │ │ └─ checkout.routes.js
    - │ │ └─ order.routes.js
    - │ │ └─ product.routes.js
    - │ │ └─ protected.routes.js
- └─ uploads/

## Környezet és függőségek

### A backend működéséhez szükséges:

- Node.js
- npm / yarn
- .env fájl, amely tartalmazza többek között:PORT=3000  
JWT\_SECRET=sajat\_titkos\_kulcs
- package.json-ben az alapvető modulok: {  
 "dependencies": {  
 "express": "^4.18.2",  
 "cors": "^2.8.5",  
 "dotenv": "^16.0.0",  
 "sequelize": "^6.32.1",  
 "bcrypt": "^5.0.1",  
 "jsonwebtoken": "^8.5.1"  
 }  
}

### Telepítés:

cd backend

npm install

### Indítás:

npm start

# vagy

node index.js

### index.js – a szerver indítása

A fájl beállítja az általános middleware-eket, definíciókat ad a route-oknak, és elindítja a szerveret.

```
const express = require('express');
```

```
const cors = require('cors');
```

```
require('dotenv').config();
```

```

const app = express();
app.use(cors());
app.use(express.json());

app.get('/', (req, res) => {
 res.send('Backend alap működik');
});

app.use('/api/auth', require('./src/routes/auth.routes'));
app.use('/api', require('./src/routes/protected.routes'));
app.use('/api/cart', require('./src/routes/cart.routes'));
app.use('/api/orders', require('./src/routes/order.routes'));
app.use('/api/checkout', require('./src/routes/checkout.routes'));
app.use('/api/products', require('./src/routes/product.routes'));
app.use('/uploads', express.static('uploads'));

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
 console.log(`Backend fut: http://localhost:${PORT}`);
});

```

## Middleware-ek

- CORS lehetővé teszi a böngészőből történő kéréseket.
- `express.json()` beállítja a JSON body-parser-t.
- A `protected`, `cart`, `order`, `checkout` és `products` útvonalakat saját routerekre delegáljuk.

## Route-ok

Minden routes/ fájl Express routert exportál. Az alábbi táblázat gyors áttekintést ad:

| Útvonal              | Metódus | Middleware     | Kontroller                    | Funkció          |
|----------------------|---------|----------------|-------------------------------|------------------|
| /api/auth/login      | POST    | –              | loginController.login         | Bejelentkezés    |
| /api/auth/register   | POST    | –              | registerController.register   | Regisztráció     |
| /api/profile         | GET     | authMiddleware | inline                        | Védett profil    |
| /api/cart/add        | POST    | optionalAuth   | cartController.addToCart      | Kosárhozáadás    |
| /api/cart            | GET     | optionalAuth   | cartController.getCart        | Kosár lekérdezés |
| /api/cart/remove     | DELETE  | optionalAuth   | cartController.removeFromCart | Eltávolítás      |
| /api/checkout        | POST    | optionalAuth   | checkoutController.checkout   | Fizetés          |
| /api/orders/checkout | POST    | optionalAuth   | orderController.checkout      | Rendelés lezárás |
| /api/products        | GET     | –              | inline                        | Terméklista      |
| /api/products/:id    | GET     | –              | inline                        | Termék részlet   |

auth.routes.js

```
const express = require('express');
```

```
const router = express.Router();
```

```
const loginController = require('../controllers/login.controller');
```

```
const registerController = require('../controllers/register.controller');
```

```
router.post('/login', loginController.login);
```

```
router.post('/register', registerController.register);
```

```
module.exports = router;
```

### protected.routes.js

```
const express = require('express');
```

```
const router = express.Router();
```

```
const authMiddleware = require('../middleware/auth.middleware');
```

```
router.get('/profile', authMiddleware, (req, res) => {
```

```
 res.json({
```

```
 message: "Sikeres belépés védett route-ra",
```

```
 user: req.user
```

```
 });
```

```
});
```

```
module.exports = router;
```

A többi route hasonló szerkezetű, a kontroller-hivatkozásokkal.

### Kontroller-ek (üzleti logika)

Ezek a funkciók dolgozzák fel a bejövő kéréseket, olvassák a body/továbbítják az adatbázis-hívásokat és formázzák a válaszokat.

### login.controller.js

```
const db = require('../models');
```

```
const User = db.User;
```

```
const bcrypt = require('bcrypt');
```

```
const jwt = require('jsonwebtoken');
```

```
exports.login = async (req, res) => {
```

```
 const { username, password } = req.body;
```

```

if (!username || !password) {
 return res.status(400).json({ message: 'Hiányzó adatok' });
}
try {
 const user = await User.findOne({ where: { username } });
 if (!user) {
 return res.status(404).json({ message: 'Felhasználó nem található' });
 }
 const passwordMatch = await bcrypt.compare(password, user.password);
 if (!passwordMatch) {
 return res.status(401).json({ message: 'Hibás jelszó' });
 }
 const token = jwt.sign(
 { id: user.user_id, username: user.username },
 process.env.JWT_SECRET,
 { expiresIn: '1h' }
);
 res.status(200).json({ message: 'Sikeres bejelentkezés', token });
} catch (error) {
 res.status(500).json({ message: 'Szerver hiba', error: error.message });
}
};

```

### register.controller.js

Validáció (minimum hossz, kis-nagybetű, szám), duplikáció-ellenőrzés, jelszó hash-elése.

### cart.controller.js

Teljes kosár-logika: vendég és bejelentkezett felhasználó egyaránt használhatja, készlet-ellenőrzés, mennyiség-növelés/csökkentés, session-id vagy user-id tárolás.



## checkout.controller.js és order.controller.js

Lényegében ugyanazt a folyamatot kezelik: a kosárban lévő tételek lekérése, végösszeg számítása, megrendelés rögzítése. A checkout controller általában fizetési logikát is tartalmazhat (pl. 3rd party API-hívás), a jelenlegi kódban egyszerű struktúra szerepel.

A pontos kód a checkout.controller.js áttekintésénél található.

## Middleware-ek

### auth.middleware.js

Token-ellenőrzés, JWT-dekódolás, a felhasználó adatait a req.user-be helyezi. Hibás vagy hiányzó token esetén 401/403 as választ ad.

```
const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
 const authHeader = req.headers.authorization;
 if (!authHeader) {
 return res.status(401).json({ message: "Nincs token megadva" });
 }
 const token = authHeader.split(' ')[1];
 try {
 const decoded = jwt.verify(token, process.env.JWT_SECRET);
 req.user = decoded;
 next();
 } catch (err) {
 return res.status(403).json({ message: "Érvénytelen token" });
 }
};
```

### optionalAuth.middleware.js

Ha van token, dekódolja, de hibás token esetén sem szakítja meg a kérés feldolgozását; vendégként (guest) továbbengedi.

## Modellek (Sequelize)

A modellek definiálják az entitásokat az ORM számára. Bár nem dokumentáljuk az adatbázis-sémát, a mezők és kapcsolatfelépítés alapján könnyen megérthető a struktúra.

### user.model.js

```
module.exports = (sequelize, DataTypes) => {
 const User = sequelize.define('User', {
 user_id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true },
 username: { type: DataTypes.STRING(50), allowNull: false, unique: true },
 email: { type: DataTypes.STRING(100), allowNull: false, unique: true },
 password: { type: DataTypes.STRING(255), allowNull: false },
 created_at: { type: DataTypes.DATE, defaultValue: DataTypes.NOW }
 }, { tableName: 'users', timestamps: false });
 return User;
};
```

A többi modell (termékek, kosártételek, megrendelések stb.) hasonlóan definiált, és szükség esetén kapcsolatok jönnek létre associate metódussal.

### index.js (modellek inicializálása)

A konfigurációt (config/config.json) olvassa, létrehozza a Sequelize példányt, majd betölti az összes modellt a könyvtárból.

## Tipikus működési folyamat

1. Felhasználó regisztrál → POST /api/auth/register → register.controller → bcrypt hashing → User.create() → vissza JSON.
2. Bejelentkezés → POST /api/auth/login → login.controller → ellenőrzés → JWT generálás.
3. Termékek listázása (nyilvános) → GET /api/products utasítással a product.routes.js kezeli a lekérdezést, opciókkal (category, limit, sort, search).
4. Kosárba helyezés:
  - a. Vendég: nincs token → optionalAuth hozzáad egy req.user-t null-lal → a cart.controller session-id-t olvassa a fejlécből.

- b. Regisztrált: JWT token a kérelemben → req.user.id alapján működik.
- 5. Checkout és Rendelés hasonlóan az aktuális kosárállapotot dolgozza fel; új rekordot hoz létre és csökkenti a készletet a termékeken.
- 6. Felhasználói profil lekérése (védett útvonal) az auth.middleware-en keresztül.

## Egyéb fontos részek

- uploads/ → a statikus fájlok (pl. termékképek) elérésének helye.
- config/config.json – adatbázis-konfigurációk környezeti beállítások szerint.
- Error handling – minden kontroller catch-blokkokkal rendelkezik, a legtöbb esetben 500-as hibát küld vissza a kliensnek a error.message-lel.

## Tesztelés / futtatás

Eszközök:

- Postman – REST API hívások kipróbálásához. Mivel a fejlesztés során Postman-t használtuk a teszteléshez, részletesen leírjuk ennek a folyamatát. Postman egy grafikus eszköz, amely lehetővé teszi HTTP kérések küldését és válaszok elemzését anélkül, hogy teljes frontendet kellene írni. Ez különösen hasznos fejlesztési fázisban, ahol gyorsan lehet ellenőrizni a backend logikát.

Postman telepítése után új collection-t lehet létrehozni a projekt számára. Minden végpontot külön request-ként lehet hozzáadni, ahol beállítható a metódus (GET, POST, stb.), URL, headers és body.

Például a regisztráció teszteléséhez:

- Új request létrehozása: POST <http://localhost:3000/api/auth/register>
- Headers: Content-Type: application/json
- Body (raw JSON):

```
{"username": "tesztfelhasznalo", "email": "teszt@pelda.hu", "password": "Jelszo123"}
```
- Küldés után ellenőrizni a válasz státuszát (200 OK esetén siker), és a JSON válaszban megjelenő üzenetet.

Bejelentkezéshez hasonlóan: POST /api/auth/login, body-ban username és password, válaszban a token-t el lehet menteni egy environment változóba (pl. "auth\_token"), hogy később használhassuk védett végpontoknál.

Kosár műveletekhez: A `/api/cart/add` esetében headers-ben hozzáadni az `Authorization: Bearer {{auth_token}}` (ha bejelentkezett felhasználó), vagy `x-session-id: valami_session_id` (vendég esetén). Body-ban `product_id` és `quantity`.

Termékek lekérdezéséhez: `GET /api/products`, query paraméterekkel mint `?search=domino&limit=5`, hogy tesztelni lehessen a szűrést és rendezést.

Postman-ben environment-eket lehet használni (pl. "dev" environment localhost-tal), és tests-et írni a válaszok automatikus ellenőrzésére, például hogy a státusz 200-e, vagy hogy a válasz tartalmaz-e bizonyos mezőket. Ez segít a regressziós tesztelésben, ahol minden változtatás után újra lehet futtatni a collection-t.

Ha hiba történik (pl. 400-as válasz hiányzó adatok miatt), Postman megjeleníti a részleteket, és könnyen lehet debug-olni a request-et módosítva.

- curl parancsok:

```
curl -X POST http://localhost:3000/api/auth/register \
-H "Content-Type: application/json" \
-d '{"username":"teszt","email":"teszt@pelda.hu","password":"Jelszo123"}'
```

```
curl http://localhost:3000/api/products?search=domino&limit=10
```

- `npm run dev` – ha van nodemon beállítva.

## Jövőbeli bővítések

- A kulcsok tényleges kiküldése a vásárló email címére
- Fizetés (`payment_method`)
- Ideiglenes leárazások/kuponkódok

## Példa kódrészletek a dokumentációhoz

### Kosárba tétel meghívása

```
await fetch('/api/cart/add', {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 'Authorization': 'Bearer ' + token,
 'x-session-id': sessionId // vendég esetén
```

```
},
 body: JSON.stringify({ product_id: 123, quantity: 2 })
});
```

### JWT ellenőrzése middleware-ben

```
const auth = require('../middleware/auth.middleware');

app.get('/api/profile', auth, (req, res) => {
 res.json(req.user);
});
```

### Összefoglalás

A backend egy klasszikus, jól szervezett Express/Sequelize alkalmazás. A kód olvasható, rétegezett, és könnyen bővíthető. A dokumentáció tartalmazza az összes releváns komponens magyarázatát, több tucat kódrészletet és használati példát, különös tekintettel a Postman tesztelési folyamatára, mivel ez volt a fejlesztés során használt eszköz.

## MCM Adatbázis Dokumentáció

### Bevezetés

Az mcm adatbázis egy teljes értékű e-commerce rendszer magja, amely számítógép hardverek, videójáték kulcsok és ajándékkártyák értékesítésére lett tervezve. Az adatbázis MariaDB 10.4.32-es verzióban működik UTF-8 karakterkódolással. A rendszer támogatja a felhasználói regisztrációt, szállítmánykövetést, kosár-kezelést, valamint rendelési folyamatokat. Az adatbázis 7 fő táblából áll, amelyek között erős relációs kapcsolatok vannak.

## Adatbázis Szerkezete - Áttekintés

Az mcm adatbázis moduláris felépítésű, ahol minden tábla egy-egy meghatározott funkciót lát el:

- users: Felhasználói fiókok és hitelesítés
- products: Az összes eladandó termék nyilvántartása
- cart\_items: Felhasználók vásárlási kosarában lévő tárgyak
- orders: Rendeléseket és azok státuszát tenyegetik
- orderdetails: Rendelt termékek részletei
- gamekeys: Videójáték kulcsok készlete
- giftcards: Ajándék kártyák készlete

Ez a felépítés biztosítja az adatok konzisztenciáját és az adatbázis integrációját a rendszer összes komponensén keresztül.

## Az egyes táblák részletes leírása

### 1. Users Tábla - Felhasználó Kezelés

A users tábla tartalmazza az összes regisztrált felhasználó adatait. Ez az adatbázis erkölcsi alapja, amelyről számos más tábla hivatkozik.

sql

```
CREATE TABLE `users` (
 `user_id` int(11) NOT NULL,
 `username` varchar(50) NOT NULL,
 `email` varchar(100) NOT NULL,
 `password` varchar(255) NOT NULL,
 `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

### Mezők magyarázata:

- user\_id: Elsődleges kulcs, egyedi felhasználó azonosító. Auto-increment típusú, automatikusan növekszik új felhasználó hozzáadásakor.
- username: Felhasználónév, maximum 50 karakter. Egyedi mező, nem ismétlődhet az adatbázisban.
- email: Felhasználó e-mail címe, maximum 100 karakter. Szintén egyedi, minden felhasználónak másik e-mail kell.
- password: Jelszó, amely bcrypt algoritmussal titkosított. 255 karakter hosszú (szükséges a hasheléshez).
- created\_at: Automatikus időbélyeg, a felhasználó regisztrációjának dátuma.

### Jelenlegi adatok:

Az adatbázisban 3 aktív felhasználó van: simon, marcell és hunor. Mindegyikük rendelkezik bcrypt titkosított jelszóval.

### Lekérdezés példa - Felhasználó keresése:

sql

```
SELECT user_id, username, email, created_at
FROM users
WHERE username = 'simon';
```

Ez a lekérdezés megtalálja simon felhasználót az összes adatával. Az output:

```
user_id: 9
username: simon
email: simon@gmail.com
created_at: 2026-03-04 10:17:30
```

## 2. Products Tábla - Termék Centrális Nyilvántartása

A products tábla az e-commerce katalógus szíve. Itt tárgy minden eladandó termék metaadatai, függetlenül a típusától.

sql

```
CREATE TABLE `products` (
 `product_id` int(11) NOT NULL,
 `name` varchar(100) NOT NULL,
 `category` enum('hardware','gamekey','giftcard') NOT NULL,
 `requires_login` tinyint(1) DEFAULT 0,
 `price` decimal(10,2) NOT NULL,
 `stock` int(11) DEFAULT 0,
 `description` text DEFAULT NULL,
 `image_url` varchar(255) DEFAULT NULL,
 `is_active` tinyint(1) DEFAULT 1,
 `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

#### Mezők magyarázata:

- `product_id`: Elsődleges kulcs, termék azonosító. Auto-increment típusú.
- `name`: Termék neve, maximum 100 karakter. Ilyenkor például: "Grand Theft Auto V Steam Key" vagy "NVIDIA GeForce RTX 4080 SUPER 16GB".
- `category`: ENUM típusú, csak három érték lehetséges: 'hardware', 'gamekey', vagy 'giftcard'. Ez meghatározza a termék típusát.
- `requires_login`: Booleán mező (0 vagy 1). Ha 1, akkor a termék csak bejelentkezett felhasználók számára vásárolható meg. Tipikusan a hardver termékeinél 1, a játékkulcsoknál és ajándékkártyáknál 0.
- `price`: Termék ára forintban, decimal típus (10,2 formátum = maximum 99999999.99). Például 5990.00.
- `stock`: Készletben levő termékek darabszáma. Ha 0, a termék elfogyott.
- `description`: Termék leírása, tetszőleges hosszúságú szöveg.
- `image_url`: A termék képe link, például "rtx4080\_super.jpg".
- `is_active`: Aktivitási státusz (0 vagy 1). Ha 0, a termék nem jelenik meg az üzletben.
- `created_at`: Időbélyeg a termék létrehozásakor.

#### Kategória szerinti termékekszám:

- Hardware: 26 termék (processzorok, videokártyák, monitorok, periféria)
- Gamekey: 25 termék (játék kulcsok)
- Giftcard: 24 termék (ajándékkártyák és előfizetések)



Lekérdezés példa - Hardver termékek 50000 Ft alatti árral:

```
sql
SELECT product_id, name, price, stock
FROM products
WHERE category = 'hardware'
AND price < 50000
AND is_active = 1
ORDER BY price DESC;
```

Ez a lekérdezés megmutatja az elérhető hardver termékeket kedvezményes áron.

Kimenete:

```
product_id: 27 | name: Wireless 26000 DPI gaming mouse | price: 22000.00 | stock: 20
product_id: 26 | name: Mechanical RGB gaming keyboard | price: 28000.00 | stock: 20
product_id: 17 | name: 1TB PCIe 4.0 NVMe SSD | price: 28000.00 | stock: 20
```

### 3. Users és Cart\_Items Reláció - Vásárlási Kosár

A cart\_items tábla egy összekötő pont a users és products között. Reprezentálja azt, hogy mely termékek állnak a felhasználó kosarában.

```
sql
CREATE TABLE `cart_items` (
 `cart_item_id` int(11) NOT NULL,
 `user_id` int(11) DEFAULT NULL,
 `session_id` varchar(255) DEFAULT NULL,
 `product_id` int(11) NOT NULL,
 `quantity` int(11) DEFAULT 1,
 `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

ALTER TABLE `cart_items`
```

```
ADD CONSTRAINT `cart_items_ibfk_1` FOREIGN KEY (`user_id`)
REFERENCES `users` (`user_id`) ON DELETE CASCADE,
ADD CONSTRAINT `cart_items_ibfk_2` FOREIGN KEY (`product_id`)
REFERENCES `products` (`product_id`) ON DELETE CASCADE;
```

Mezők magyarázata:

- `cart_item_id`: Elsődleges kulcs, kosárelem azonosító.
- `user_id`: Idegen kulcs, a `users` táblára hivatkozik. A bejelentkezett felhasználó azonosítója.
- `session_id`: Munkamenet azonosító vendég felhasználók számára (`varchar 255`). Ha `user_id` NULL, akkor egy vendég kosarából van szó.
- `product_id`: Idegen kulcs, a termék azonosítója.
- `quantity`: Darabszám, hány darab termék van a kosarban (alapértelmezés: 1).
- `created_at`: Quando lett a termék a kosarba helyezve.

Fontos: Az ON DELETE CASCADE meghatározás azt jelenti, ha egy termék vagy felhasználó törlésre kerül, az összes hozzátartozó kosárelem is automatikusan törlődik.

Lekérdezés példa - Simon kosarának tartalma:

sql

```
SELECT ci.cart_item_id, p.name, ci.quantity, p.price,
 (p.price * ci.quantity) AS total
FROM cart_items ci
JOIN products p ON ci.product_id = p.product_id
WHERE ci.user_id = 9;
```

Ez a lekérdezés simon (9-es felhasználó) kosarában lévő összes terméket megmutatja az adott darabszámokkal és árakkal.

## 4. Orders tábla - rendelések kezelése

Az `orders` tábla a vásárlások főrekordja. Egy rendelés szerzői információt, fizetési adatokat és szállítási címet tartalmaz.

sql

```
CREATE TABLE `orders` (
 `order_id` int(11) NOT NULL,
 `user_id` int(11) DEFAULT NULL,
 `session_id` varchar(255) DEFAULT NULL,
 `order_date` timestamp NOT NULL DEFAULT current_timestamp(),
 `status` enum('pending','paid','shipped','completed','cancelled') DEFAULT 'pending',
 `payment_method` enum('card','paypal','bank','none') DEFAULT 'none',
 `payment_id` varchar(100) DEFAULT NULL,
 `total_amount` decimal(10,2) DEFAULT 0.00,
 `first_name` varchar(255) DEFAULT NULL,
 `last_name` varchar(255) DEFAULT NULL,
 `email` varchar(150) NOT NULL,
 `phone` varchar(255) DEFAULT NULL,
 `zip_code` varchar(20) DEFAULT NULL,
 `city` varchar(255) DEFAULT NULL,
 `address_line` varchar(255) DEFAULT NULL,
 `additional_info` varchar(255) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
ALTER TABLE `orders`
 ADD CONSTRAINT `orders_ibfk_1` FOREIGN KEY (`user_id`)
 REFERENCES `users` (`user_id`) ON DELETE CASCADE;
```

#### Mezők magyarázata:

- `order_id`: Elsődleges kulcs, rendelés azonosító.
- `user_id`: Idegen kulcs, bejelentkezett felhasználó. NULL lehet vendégek rendelésekor.
- `session_id`: Munkamenet azonosító vendég vásárlásokhoz. NULL a bejelentkezett felhasználóknál.
- `order_date`: Rendelés időpontja, automatikus időbélyeg.
- `status`: ENUM mező, 5 lehetséges állapot:
  - `'pending'`: Kivárákozik a fizetésre
  - `'paid'`: Fizetett, feldolgozásra vár
  - `'shipped'`: Elküldve

- 'completed': Teljesítve
- 'cancelled': Lemondva
- payment\_method: ENUM, fizetési módszer: 'card' (kártya), 'paypal', 'bank' (banki átutalás), vagy 'none' (nincs feldolgozva).
- payment\_id: Az írás kiszolgáló fizetési tranzakció azonosítója.
- total\_amount: A teljes rendelés összege (tizedesponnttal).
- first\_name, last\_name: A vásárló neve.
- email: A vásárló e-mail címe.
- phone: Telefonszám.
- zip\_code: Irányítószám.
- city: Város.
- address\_line: Utca és házszám.
- additional\_info: Megjegyzések, például porta szám, vagy emelet.

Jelenlegi rendelések:

Az adatbázisban 4 rendelés van, mindegyik "pending" (függőben) állapotban.

#### Lekérdezés példa - Összes befejezett rendelés:

sql

```
SELECT order_id, user_id, order_date, total_amount,
 CONCAT(first_name, ' ', last_name) AS customer_name, city
FROM orders
WHERE status = 'completed'
ORDER BY order_date DESC;
```

Ez a lekérdezés a befejezett rendeléseket mutatja időpontok szerinti csökkenő sorrendben. Hasznos a vásárlási történet követésére.

## 5. OrderDetails tábla - rendelés sorok

Az orderdetails tábla az orders táblá bontópontja, amely soronként táryn meg, mely termékek voltak az adott rendelésben.

sql

```
CREATE TABLE `orderdetails` (
 `detail_id` int(11) NOT NULL,
 `order_id` int(11) NOT NULL,
 `product_id` int(11) NOT NULL,
 `quantity` int(11) DEFAULT 1,
 `price_at_purchase` decimal(10,2) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
ALTER TABLE `orderdetails`
 ADD CONSTRAINT `orderdetails_ibfk_1` FOREIGN KEY (`order_id`)
 REFERENCES `orders` (`order_id`) ON DELETE CASCADE,
 ADD CONSTRAINT `orderdetails_ibfk_2` FOREIGN KEY (`product_id`)
 REFERENCES `products` (`product_id`);
```

#### Mezők magyarázata:

- detail\_id: Elsődleges kulcs, rendelés sor azonosító.
- order\_id: Idegen kulcs az orders táblára.
- product\_id: Idegen kulcs a products táblára.
- quantity: Az adott termék mennyisége az adott rendelésben.
- price\_at\_purchase: A termék ára a vásárlás időpontjában. Ez fontos, mert a products táblában lévő ár változhat.

#### Lekérdezés példa - 4-es rendelés részletei:

sql

```
SELECT od.detail_id, p.name, od.quantity, od.price_at_purchase,
 (od.quantity * od.price_at_purchase) AS line_total
FROM orderdetails od
JOIN products p ON od.product_id = p.product_id
WHERE od.order_id = 4;
```

#### Output:

detail\_id: 4 | name: Ryzen 7 5800X | quantity: 1 | price\_at\_purchase: 85000.00 | line\_total:  
85000.00

## 6. GameKeys Tábla - videójáték kulcsok nyilvántartása

A gamekeys tábla a videójáték kulcsok konkrét adatait tárolja meg. Minden játékkulcs egy unique kódot tartalmaz.

sql

```
CREATE TABLE `gamekeys` (
 `key_id` int(11) NOT NULL,
 `product_id` int(11) NOT NULL,
 `game_name` varchar(100) NOT NULL,
 `platform` varchar(50) NOT NULL,
 `code` varchar(100) NOT NULL,
 `is_used` tinyint(1) DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
ALTER TABLE `gamekeys`
 ADD CONSTRAINT `gamekeys_ibfk_1` FOREIGN KEY (`product_id`)
 REFERENCES `products` (`product_id`) ON DELETE CASCADE;
```

### Mezők magyarázata:

- key\_id: Elsődleges kulcs, kulcs azonosító.
- product\_id: Idegen kulcs, melyik termékhez tartozik.
- game\_name: A játék neve, például "Cyberpunk 2077".
- platform: A platform, ahol érvényes: "Steam", "Rockstar", "Xbox", "BattleNet", "Epic", stb.
- code: A 100 karakteres unique kód. Egyedi az adatbázisban (UNIQUE KEY).
- is\_used: Booleán (0 vagy 1). 1 ha már felhasználta valaki, 0 ha még elérhető

### Platformok az adatbázisban:

- Steam (a leggyakoribb)
- Rockstar
- Xbox
- EA
- Ubisoft

- BattleNet
- Epic
- Riot
- PlayStation
- GOG
- Mojang

Lekérdezés példa - Elérhető GTA V kulcsok:

sql

```
SELECT key_id, product_id, game_name, platform, code
FROM gamekeys
WHERE game_name = 'Grand Theft Auto V'
AND is_used = 0;
```

Ez megmutatja az összes nem használt GTA V kulcsot, amelyet az üzletben el lehet adni.

## 7. GiftCards tábla - ajándékkártyák kezelése

A giftcards tábla az ajándékkártyák és előfizetési vonalak nyilvántartása.

sql

```
CREATE TABLE `giftcards` (
 `card_id` int(11) NOT NULL,
 `product_id` int(11) NOT NULL,
 `platform` varchar(50) NOT NULL,
 `value` decimal(10,2) NOT NULL,
 `code` varchar(100) NOT NULL,
 `is_used` tinyint(1) DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
ALTER TABLE `giftcards`
 ADD CONSTRAINT `giftcards_ibfk_1` FOREIGN KEY (`product_id`)
 REFERENCES `products` (`product_id`) ON DELETE CASCADE;
```

### Mezők magyarázata:

- card\_id: Elsődleges kulcs, kártya azonosító.
- product\_id: Idegen kulcs, melyik termékhez tartozik.
- platform: A platform neve: "Steam", "Xbox", "PlayStation", "Nintendo", "EA", "Netflix", "Spotify", stb.
- value: A kártya értéke euróban vagy nullás érték az előfizetésekhez. Decimal típus.
- code: A megváltható kód. Egyedi (UNIQUE KEY).
- is\_used: Booleán, igaz ha már megváltották.

### Lekérdezés példa - Elérhető Steam ajándékkártyák:

sql

```
SELECT card_id, platform, value, code
FROM giftcards
WHERE platform = 'Steam'
AND is_used = 0;
```

### Output (értékek szerint csoportosítva):

```
card_id: 1 | platform: Steam | value: 20.00 | code: STEAM-CARD-001
card_id: 2 | platform: Steam | value: 50.00 | code: STEAM-CARD-002
card_id: 25 | platform: Steam | value: 100.00 | code: STEAM-CARD-003
```

## Adatbázis Relációk Összefoglalása

### Az adatbázis viszonyai így néznek ki:

users (1) — (N) cart\_items  
          |— (N) orders

products (1) — (N) cart\_items  
          |— (N) orderdetails  
          |— (N) gamekeys  
          └— (N) giftcards

orders (1) — (N) orderdetails



- 1:N relációk: Egy users record több cart\_items és orders rekorddal lehet kapcsolatban.
- Kaskádos törlés: Ha egy users vagy products törlésre kerül, az összes hozzátartozó rekord is törlődik.

## Teljes Rendelés (Order + Details + Product Info)

sql

```
SELECT
 o.order_id,
 o.order_date,
 CONCAT(o.first_name, ' ', o.last_name) AS customer,
 o.email,
 p.name AS product_name,
 od.quantity,
 od.price_at_purchase,
 (od.quantity * od.price_at_purchase) AS line_total,
 o.total_amount,
 o.status
FROM orders o
LEFT JOIN orderdetails od ON o.order_id = od.order_id
LEFT JOIN products p ON od.product_id = p.product_id
ORDER BY o.order_date DESC;
```

Ez a lekérdezés összes rendelést megmutatja minden részlettel, ideális a rendelések áttekintéséhez.

## Felhasználó vásárlási története

```
sql
SELECT
 u.username,
 COUNT(o.order_id) AS total_orders,
 SUM(o.total_amount) AS total_spent,
 MAX(o.order_date) AS last_order_date
FROM users u
LEFT JOIN orders o ON u.user_id = o.user_id
GROUP BY u.user_id, u.username
ORDER BY total_spent DESC;
```

Ez a lekérdezés minden felhasználóra mutatja a rendelések számát és teljes költségét.

## Készlet nyilvántartás (hardware + kulcsok + kártyák)

```
sql
SELECT
 p.product_id,
 p.name,
 p.category,
 p.price,
 p.stock,
 CASE
 WHEN p.category = 'gamekey' THEN
 (SELECT COUNT(*) FROM gamekeys WHERE product_id = p.product_id AND is_used
= 0)
 WHEN p.category = 'giftcard' THEN
 (SELECT COUNT(*) FROM giftcards WHERE product_id = p.product_id AND is_used
= 0)
 ELSE p.stock
```

```
END AS available_units
FROM products p
WHERE p.is_active = 1
ORDER BY p.category, p.price;
```

Ez megmutatja az összes elérhető termék készletét, akár hardver, akár digitális termék.

## Teljesítmény optimalizálás és indexek

Az adatbázisban az alábbi indexek vannak definiálva:

1. Elsődleges kulcsok (PRIMARY KEY): Minden táblán egyedi azonosítást biztosítanak.
2. Idegen kulcsok (FOREIGN KEY): A relacionális integritást biztosítják.
3. Egyedi indexek (UNIQUE KEY):
4. `username` (users)
5. `email` (users, orders)
6. `code` (gamekeys, giftcards)

Ezek az indexek biztosítják a gyors lekérdezéseket és az adatok konzisztenciáját.

## Biztonsági megjegyzések

1. Jelszó titkosítás: Az users táblában az összes jelszó bcrypt algoritmussal van titkosítva (2b\$10\$ hash formato).
2. UTF-8 kódolás: Az adatbázis UTF-8mb4 karakterkódolást használ, amely támogatja az összes különleges karaktert és emoji-t.
3. Felhasználói adatok: Az orders táblában személyes összegek óvottan vannak tárolva.

## Jelenlegi adatok összefoglalása

- Felhasználók: 3 (simon, marcell, hunor)
- Termékek: 75 aktív termék
- Hardware: 26

- Játékkulcsok: 25
- Ajándékkártyák: 24
- Rendelések: 4 függőben lévő rendelés
- Rendelés sorok: 5
- Játékkulcsok: 25 (mind felhasználatlan)
- Ajándékkártyák: 25 (mind felhasználatlan)

## Források

### Technológiák és Keretrendszerek

#### Backend:

- Node.js: JavaScript runtime környezet a szerveroldali alkalmazáshoz
- Express.js: Web alkalmazás keretrendszer REST API-k létrehozására
- Sequelize: Promise-alapú Node.js ORM MySQL adatbázisokhoz
- JWT (JSON Web Tokens): Biztonságos felhasználói autentikáció és autorizáció
- bcrypt: Jelszavak biztonságos titkosítása

#### Frontend:

- Angular 17.3.0: TypeScript-alapú web alkalmazás keretrendszer
- Angular Material: UI komponens könyvtár modern, responszív felületekhez
- RxJS: Reaktív programozási könyvtár aszinkron műveletek kezelésére

#### Adatbázis:

- MariaDB 10.4.32: Relációs adatbázis-kezelő rendszer MySQL kompatibilitással
- MySQL2: Node.js driver MariaDB/MySQL kapcsolatokhoz

### Programozási Nyelvek

- JavaScript (ES6+): Backend logika és API végpontok
- TypeScript 5.4.2: Frontend fejlesztés típusbiztonsággal
- SQL: Adatbázis lekérdezések és séma definíciók

### Fejlesztési Eszközök és Könyvtárak

#### Backend Dependencies:

- cors: Cross-Origin Resource Sharing engedélyezése
- dotenv: Környezeti változók kezelése

- mysql2: Adatbázis kapcsolatok
- sequelize: Adatbázis modellek és migrációk
- sequelize-cli: Parancssori eszköz adatbázis műveletekhez
- jsonwebtoken: Token alapú autentikáció
- bcrypt: Jelszó hashelés

## Frontend Dependencies:

- @angular/core, @angular/common, @angular/router: Angular alap komponensek
- @angular/material, @angular/cdk: UI komponensek és fejlesztési eszközök
- @angular/forms: Reaktív és template-alapú űrlapok
- @angular/animations: Animációk támogatása
- rxjs: Objektum-orientált reaktív programozás
- zone.js: Aszinkron műveletek kezelése

## Fejlesztési Eszközök:

- Angular CLI: Projekt generálás, build és fejlesztési szerver
- Nodemon: Automatikus szerver újraindítás fejlesztéskor
- TypeScript Compiler: TS kód JavaScript-re fordítása
- Jasmine & Karma: Unit tesztelés frontend számára

## Projekt Struktúra és Konfiguráció

### Backend Struktúra:

- src/controllers/: Üzleti logika kezelők (auth, cart, checkout, order, product)
- src/models/: Sequelize adatbázis modellek
- src/routes/: API végpont definíciók
- src/middleware/: Hitelesítés és autorizáció middleware-ek
- database/: SQL dump és adatbázis séma
- uploads/: Feltöltött fájlok tárolása

### Frontend Struktúra:

- src/app/features/: Funkcionális modulok (auth, cart, checkout, home, orders, products, profile)
- src/app/services/: HTTP szolgáltatások API kommunikációhoz
- src/app/models/: TypeScript interfészek és típusdefiníciók
- src/app/shared/: Megosztott komponensek (header, cart-drawer, toast)
- src/app/core/: Alkalmazás szintű szolgáltatások (auth interceptor)

## Környezeti Konfiguráció

- Backend: .env fájl környezeti változókhoz (adatbázis kapcsolat, JWT secret, port)
- Frontend: proxy.conf.json fejlesztési proxy konfiguráció API hívásokhoz
- Adatbázis: UTF-8mb4 karakterkódolás teljes Unicode támogatásért

## Verziókezelés és Deployment

- Git: Verziókezelés
- npm: Package management mindkét oldalon
- Node.js 18+: Minimum követelmény a backend futtatásához
- Angular 17+: Frontend keretrendszer verziója